

Aula 05 — Arquiteturas (parte 3)

Resumo 6 de 14 · arquivo aula05 · 21 imagens / 20 slides · Cap. 2 de van Steen & Tanenbaum

Fecha o tema de arquiteturas com os sistemas **simetricamente distribuídos** (peer-to-peer) e os **híbridos**. Em P2P todos os processos são iguais (cada um é cliente e servidor). Vê-se P2P **estruturado** (topologia conhecida + DHT), **não estruturado** (grafo aleatório), **super-peer**, e depois nuvem, borda e blockchain.

1. Distribuição vertical, horizontal e P2P

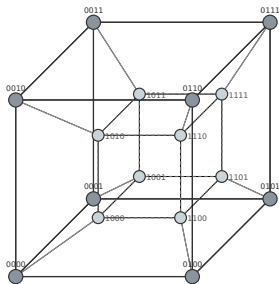
- Distribuição vertical:** dividir a aplicação em camadas lógicas e pôr cada camada numa máquina diferente (é a arquitetura em 3 camadas da aula anterior).
- Distribuição horizontal:** dividir um cliente ou servidor em partes *logicamente equivalentes*, cada uma operando sobre sua fatia do conjunto de dados.
- Peer-to-peer (P2P):** todos os processos são iguais — cada um atua como cliente e servidor ao mesmo tempo (distribuição horizontal levada ao extremo).

2. P2P estruturado

O sistema é organizado numa **topologia conhecida**, usada para localizar dados eficientemente. Emprega um **índice livre de semântica**: cada item recebe uma **chave** única (geralmente via função de hash) e o sistema armazena pares (chave, valor) — a estrutura típica é uma **DHT (distributed hash table)**.

Exemplo 1 — Hipercubo

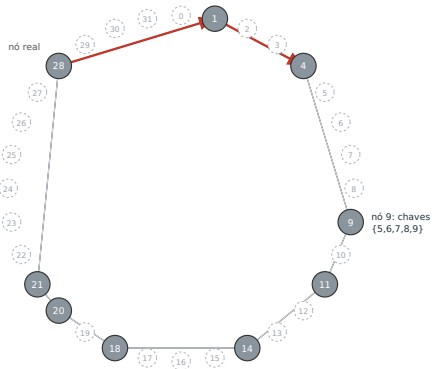
Nós recebem ids de m bits (aqui 4 bits, 16 nós); procurar a chave k significa **rotear até o nó de id k**. Vizinhos diferem em exatamente 1 bit, então cada salto corrige um bit. Desvantagem: é **difícil adicionar/remover nós**.



Hipercubo de 4 bits (16 nós): nós vizinhos diferem em 1 bit; a busca corrige um bit por salto.

Exemplo 2 — Chord

Os nós formam um **anel lógico**, cada um com id de m bits; cada item é mapeado por hash a uma chave de m bits. A chave k é guardada no nó **sucessor** — o menor id $\geq k$. O anel ganha vários **atalhos** (finger table) para acelerar a busca de $O(N)$ para $O(\log N)$.



busca(3) a partir do nó 9: 28 → 1 → 4 (nó 4 = sucessor da chave 3)

Anel Chord (m=5, 32 posições): cinza = nós reais, tracejado = ids sem nó. O nó 9 guarda as chaves {5-9}. A busca pela chave 3 salta pelos atalhos: 28 → 1 → 4.

3. P2P não estruturado

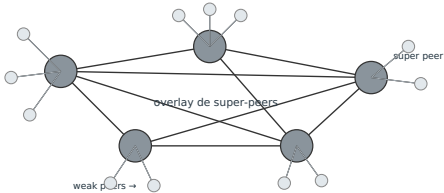
Cada nó mantém uma lista *ad hoc* de vizinhos; a rede resultante parece um **grafo aleatório**. Como não há índice, a busca é por tentativa:

- **Inundação (flooding):** o nó repassa a busca a *todos* os vizinhos; quem já viu ignora. Limitada por um TTL (número máximo de saltos). Encontra rápido, mas gera muito tráfego.
- **Passeio aleatório (random walk):** repassa a busca a *um* vizinho aleatório, que repassa a outro, e assim por diante. Mais econômico em comunicação, mas pode demorar mais.

Com N nós e cada item replicado em r nós, o passeio aleatório precisa sondar em média **$S \approx N/r$** nós. Comparação da aula: com $r/N = 0,001$, o passeio dá $S \approx 1000$; já a inundação com $d = 10$ e $k = 4$ saltos contata 7290 nós — muito mais mensagens.

4. Redes super-peer

Às vezes vale **quebrar a simetria**: alguns nós mais capazes viram **super-peers**, que formam um overlay entre si e servem de **servidores de índice** e **brokers** para os **weak peers** ligados a eles. Melhora a busca e a decisão de onde armazenar dados.



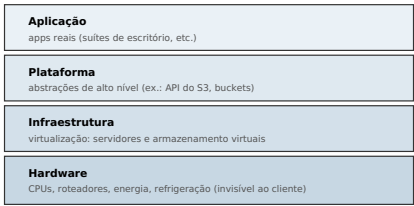
Super-peers formam um overlay entre si; cada um coordena vários weak peers (nós comuns).

Exemplo — BitTorrent

Para baixar um arquivo F: busca-se num diretório global, que devolve um **arquivo .torrent**; este aponta para um **tracker** (servidor que sabe quais nós ativos têm partes de F). O nó entra no **enxame (swarm)**, pega uma parte de graça e passa a **trocar** pedaços com outros peers — colaboração incentivada.

5. Sistemas híbridos

Computação em nuvem (4 camadas)



↑ mais abstrato - mais concreto ↓

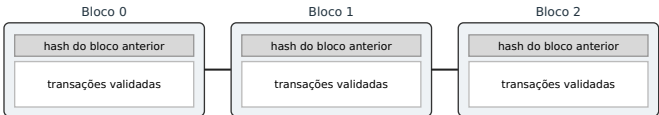
As quatro camadas da nuvem, do hardware físico (invisível ao cliente) até as aplicações.

Arquitetura borda-nuvem (edge)

Servidores colocados na **borda** da rede (fronteira entre a rede da organização e a Internet). Os argumentos comuns a favor da borda — e a ressalva da aula de que muitos são *equivocados*: **latência/banda** (importa para tempo real como AR/VR, mas muita gente subestima a nuvem), **confiabilidade** (assume-se que a nuvem é pouco confiável — em geral falso) e **segurança/privacidade** (assume-se que "perto = mais seguro" — a prática mostra que costuma ser falso). A **orquestração de borda** é mais difícil que na nuvem: exige alocação de recursos, **posicionamento** de serviços (placement) e **seleção de borda** (a mais próxima nem sempre é a melhor).

Blockchain

Blocos formam uma **cadeia append-only**; cada bloco é **imutável** e massivamente replicado. O processo: (1) um nó difunde uma transação; (2) um validador junta transações num bloco; (3) o bloco validado é difundido a todos. A real dificuldade é **quem tem permissão para anexar** um bloco — o problema de consenso:



cadeia append-only: cada bloco referencia o hash do anterior => imutável

Estrutura em cadeia: cada bloco carrega o hash do anterior, tornando o histórico imutável.

- **Centralizada:** uma única entidade decide quem anexa. Simples, mas contraria a razão de existir da blockchain.
- **Distribuída (com permissão):** um grupo pequeno de servidores chega a consenso; funciona mesmo que alguns falhem/mintam, desde que $\sim 2/3$ se comportem corretamente. Na prática, só algumas dezenas de servidores.
- **Descentralizada (sem permissão):** todos participam de uma eleição de líder; só o líder eleito anexa o bloco. Fazer essa eleição em larga escala de forma justa, robusta e segura é longe de trivial.